QBIT Hub
Enterprise SaaS Port

1. QA Methodology

This report presents the results of aggressive Monkey Testing performed on the QBIT Hub Enterprise Portal v2.0. The testing methodology included: static code analysis (grep-based pattern matching across 419 source files), production API testing (curl-based HTTP requests), build verification (TypeScript + Next.js), database schema audit, and security audit. The goal was to find every hidden bug, broken workflow, crash, and edge case.

2. Bug Summary

Severity	Count	Description
CRITICAL	1	Production deployment is stale — latest modules not deployed
HIGH	5	Missing error handling, unsafe JSON.parse, XSS risk, missing rel=noopener, stale API 404s
MEDIUM	7	Missing React keys, no AbortController, useEffect leaks, 'as never' casts, interval leaks
LOW	4	any types, TODO comments, hardcoded localhost refs, dark mode untested
TOTAL	17	All bugs documented below with steps, expected, actual, and suggested fix

QA Bug Report Monkey Testing

Enterprise Software QA — Aggressive Monkey Testing
QBIT Hub Enterprise Portal — Version 2.0

17 Bugs Found | 1 Critical | 5 High | 7 Medium | 4 Low

Document

QA Bug Report — Monkey Testing Results

Version: 2.0 | Date: 14 July 2026

Classification: Internal QA

3. Detailed Bug Report

3.1 CRITICAL Issues

Bug ID: QA-001

Severity: CRITICAL

Page: All pages (Production)

Steps: 1. Visit https://qbithub.vercel.app/?screen=fleet-manager 2. Observe page loads (HTTP 200) but API calls to /api/fleet/devices return HTML 404 3. Same for /api/analytics/executive, /api/dr-qbit/diagnostics, /api/dr-qbit/tests 4. Login as admin — authenticated API calls also return 404 HTML page

Expected: All 83 API routes should be available on production. Fleet Manager, Analytics, Dr. QBIT Diagnostics, and Test Center should return JSON data.

Actual: Only the older deployed routes work. 20+ newer routes (fleet, analytics, dr-qbit diagnostics/tests) return 404. The Vercel deployment is stale — the Vercel token expired and the latest commits (c0df1df) were never deployed.

Suggested Fix: Redeploy to Vercel: either install GitHub App for auto-deploy, or provide a new Vercel token and run `vercel deploy --prod --force`. Verify all 83 routes return JSON after deployment.

3.2 HIGH Issues

Bug ID: QA-002

Severity: HIGH

Title: 41 API routes lack try-catch error handling

Steps: Any database query failure in these 41 routes will produce an unhandled exception → 500 Internal Server Error with stack trace instead of a clean JSON error response.

Expected: All API routes should catch errors and return { error: 'message' } with appropriate HTTP status.

Actual: 41 routes (analytics/*, fleet/*, dr-qbit/*, public/*) have no try-catch. If Prisma query fails (e.g. database timeout, connection limit), server returns raw 500 error.

Suggested Fix: Wrap all route handlers in try-catch blocks (or use the existing apiHandler() wrapper from @/lib/errors/handler).

Bug ID: QA-003

Severity: HIGH

Title: 11 unsafe JSON.parse calls without try-catch in API routes

Steps: API routes call JSON.parse() on database string fields (evidence, testsPerformed, supportedOses, openPorts, supportedModels, channels). If stored data is malformed, the API crashes.

Expected: JSON.parse should be wrapped in try-catch with fallback to empty array/null.

Actual: 11 JSON.parse calls in API routes have no error handling. Corrupted JSON in DB → 500 crash.

Suggested Fix: Create a safeJsonParse() utility: try { JSON.parse(str) } catch { return fallback }

Bug ID: QA-004

Severity: HIGH

Title: XSS risk: dangerouslySetInnerHTML in MessageBubble.tsx

Steps: AI chat responses are rendered with dangerouslySetInnerHTML via formatInline(). If the AI model returns <script> tags or event handlers, they will be injected into the DOM.

Expected: All HTML rendered from AI responses should be sanitized (strip <script>, on* attributes, javascript: URLs).

Actual: formatInline() processes AI-generated text and renders it as raw HTML. No sanitization is applied.

Suggested Fix: Add DOMPurify or a custom sanitizer to strip dangerous HTML before rendering. Or use a markdown renderer (react-markdown) instead of dangerouslySetInnerHTML.

Bug ID: QA-005

Severity: HIGH

Title: 14 external links missing rel='noopener noreferrer'

Steps: Multiple components use target='_blank' without rel='noopener noreferrer'. This allows the opened page to access window.opener (tab-nabbing attack).

Expected: All target='_blank' links should include rel='noopener noreferrer'.

Actual: 14 links across YouTubeGallery, TroubleshootingSection, RelatedVideos, DeviceTestCard, EngineerActions, PhotoUploader, CustomerActions, EngineerCard, CustomerTrackingPortalPage.

Suggested Fix: Add rel='noopener noreferrer' to all target='_blank' anchor tags. Or create a SafeLink component that enforces this.

Bug ID: QA-006

Severity: HIGH

Title: Protected API returns HTML 404 instead of JSON 401 when route doesn't exist

Steps: When an unauthenticated user hits /api/fleet/devices on the stale production deployment, they get a full HTML 404 page instead of a JSON error.

Expected: API endpoints should always return JSON, even for 404/401/403 errors.

Actual: Next.js renders the 404 HTML page for non-existent API routes instead of a JSON error. This breaks client-side error handling that expects JSON.

Suggested Fix: This is a side effect of QA-001 (stale deployment). After redeploying, all routes will exist and return proper JSON errors. Additionally, add a catch-all API route that returns JSON 404.

3.3 MEDIUM Issues

Bug ID: QA-007

Severity: MEDIUM

Title: 348 .map() iterations missing key prop

Steps: React requires a unique 'key' prop on list items rendered via .map(). ~348 .map() calls were found without key= in the JSX.

Expected: All .map() iterations should have key={uniqueId} on the top-level element.

Actual: Many .map() calls are missing keys. React will log console warnings and may cause incorrect re-renders.

Suggested Fix: Audit all .map() calls and add key props. Use a stable ID (not array index) where possible.

Bug ID: QA-008

Severity: MEDIUM

Title: Zero AbortController usage — fetch requests not cancellable

Steps: When a user navigates away from a page while a fetch is in-flight, the response handler tries to update state on an unmounted component.

Expected: Fetch requests should be cancellable via AbortController. useEffect cleanup should abort pending requests.

Actual: 0 AbortController instances found. 62 useEffect calls with only 14 cleanup functions. State updates after unmount will cause React warnings.

Suggested Fix: Add AbortController to fetch calls in useEffect. Pass signal to fetch(). Abort in cleanup function.

Bug ID: QA-009

Severity: MEDIUM

Title: 48 useEffect hooks without cleanup functions

Steps: 62 useEffect calls found, only 14 have cleanup (return () => ...). This means 48 effects may leak.

Expected: All useEffect hooks with subscriptions, intervals, or async operations should return a cleanup function.

Actual: 48 effects without cleanup. Potential memory leaks, especially in pages with intervals (CustomerTrackingPortalPage auto-refresh, SyncStatus polling).

Suggested Fix: Add cleanup functions to all useEffect hooks that create intervals, subscriptions, or event listeners.

Bug ID: QA-010

Severity: MEDIUM

Title: 5 setInterval calls — some may not be cleaned up

Steps: setInterval is used in TourOverlay (500ms), SyncStatus (5000ms), WelcomeHero (60000ms), CustomerTrackingPortalPage (30000ms), ImportWizard (polling).

Expected: All intervals should be cleared in useEffect cleanup: return () => clearInterval(interval).

Actual: Need to verify each interval has a cleanup. CustomerTrackingPortalPage has cleanup (verified). Others need verification.

Suggested Fix: Audit all 5 setInterval calls and ensure clearInterval is called in the corresponding useEffect cleanup.

Bug ID: QA-011

Severity: MEDIUM

Title: 26 'as never' type casts bypass TypeScript checking

Steps: 26 instances of 'as never' found in .tsx files. These casts bypass TypeScript's type system, potentially hiding real type errors.

Expected: Use proper type annotations instead of 'as never'. If the type is truly unknown, use 'as unknown as Type' with a comment explaining why.

Actual: 'as never' is used to bypass type checking on navigate() calls and status mappings. This could hide real type mismatches.

Suggested Fix: Replace 'as never' with proper type assertions. Add the correct ScreenId type to navigation calls.

Bug ID: QA-012

Severity: MEDIUM

Title: 10 empty catch blocks silently swallow errors

Steps: 10 catch blocks found with empty bodies {} — errors are silently swallowed with no logging or user feedback.

Expected: Catch blocks should at minimum log the error (console.error) and optionally show a user-facing toast.

Actual: 10 empty catch blocks in tour-context.tsx (6), health/route.ts (1), compatibility-checker.ts (1), pwa/hooks.ts (1), offline-queue.ts (1).

Suggested Fix: Add console.error or a logging call to each empty catch block. For user-facing operations, add toast notifications.

Bug ID: QA-013

Severity: MEDIUM

Title: 5 dangerouslySetInnerHTML uses (4 in MessageBubble, 1 in chart.tsx)

Steps: Beyond the XSS risk in QA-004, chart.tsx also uses dangerouslySetInnerHTML for CSS injection.

Expected: Minimize dangerouslySetInnerHTML usage. Sanitize all dynamic content.

Actual: 4 uses in MessageBubble.tsx for AI response rendering. 1 in chart.tsx (shadcn/ui component — lower risk).

Suggested Fix: Replace MessageBubble's HTML rendering with react-markdown. Audit chart.tsx for sanitization.

3.4 LOW Issues

Bug ID: QA-014

Severity: LOW

Title: 2 'any' type annotations in source code

Steps: 1 in rbac/roles.ts (in a comment), 1 in notifications/reminders.ts (anySent variable).

Expected: No 'any' types in strict TypeScript mode.

Actual: 2 instances. Non-blocking but violates strict TypeScript convention.

Suggested Fix: Replace 'anySent' with a properly typed boolean. The comment usage is acceptable.

Bug ID: QA-015

Severity: LOW

Title: 2 TODO/FIXME comments left in code

Steps: tracking/types.ts has comments about phone masking that reference the implementation.

Expected: All TODOs should be resolved or tracked in an issue tracker.

Actual: 2 comments in tracking/types.ts. They are documentation comments, not actual TODOs.

Suggested Fix: No fix needed — these are documentation comments, not action items.

Bug ID: QA-016

Severity: LOW

Title: Dark mode toggle exists but not fully tested

Steps: Theme toggle (light/dark/system) exists in TopBar. Material 3 dark palette defined in globals.css.

Expected: Dark mode should be visually tested across all pages.

Actual: Dark mode was not tested during this QA cycle. Some pages may have contrast issues or hardcoded white backgrounds.

Suggested Fix: Perform visual testing of all 40 pages in dark mode. Fix any contrast or background issues.

Bug ID: QA-017

Severity: LOW

Title: Demo seed data in production database

Steps: Production Neon Postgres contains demo users, demo work orders, demo devices, demo notifications, demo firmware, demo test results.

Expected: Production database should contain only real data.

Actual: Demo data is present (acceptable for demo/staging, but should be cleaned before real go-live).

Suggested Fix: Before real go-live: run prisma db reset, re-seed with only real users (admin + engineers), remove demo work orders and devices.

4. Bug Summary Table

Bug ID	Severity	Title	Status
QA-001	CRITICAL	Production deployment stale (20+ routes 404)	NOT FIXED
QA-002	HIGH	41 API routes lack try-catch error handling	NOT FIXED
QA-003	HIGH	11 unsafe JSON.parse calls without try-catch	NOT FIXED
QA-004	HIGH	XSS risk: dangerouslySetInnerHTML in MessageBubble	NOT FIXED
QA-005	HIGH	14 external links missing rel=noopener	NOT FIXED
QA-006	HIGH	Protected API returns HTML 404 instead of JSON	NOT FIXED
QA-007	MEDIUM	348 .map() iterations missing key prop	NOT FIXED
QA-008	MEDIUM	Zero AbortController usage (fetch leaks)	NOT FIXED
QA-009	MEDIUM	48 useEffect hooks without cleanup	NOT FIXED
QA-010	MEDIUM	5 setInterval calls — cleanup not verified	NOT FIXED
QA-011	MEDIUM	26 'as never' type casts bypass checking	NOT FIXED
QA-012	MEDIUM	10 empty catch blocks swallow errors	NOT FIXED
QA-013	MEDIUM	5 dangerouslySetInnerHTML uses	NOT FIXED
QA-014	LOW	2 'any' type annotations	NOT FIXED
QA-015	LOW	2 TODO/FIXME comments (documentation)	NOT FIXED
QA-016	LOW	Dark mode not fully tested	NOT FIXED
QA-017	LOW	Demo seed data in production database	NOT FIXED

5. Recommendation

As instructed, no bugs have been fixed yet. This report is for review. We recommend fixing in the following priority order:

- Fix QA-001 first (redeploy to Vercel) — this is blocking all production testing.
- Fix QA-002 + QA-003 (API error handling) — prevents 500 crashes on database errors.
- Fix QA-004 (XSS sanitization) — security vulnerability in AI chat.
- Fix QA-005 (rel=noopener) — quick fix, 14 links.
- Fix QA-007 + QA-008 + QA-009 (React best practices) — prevents memory leaks.
- Low priority items can be addressed in a follow-up sprint.

Waiting for approval before fixing any issue.